Unità 3 – Teoria dei Segnali

Il controllo degli errori in trasmissione

Francesco Gobbi

29 aprile 2026

I.I.S.S. Galileo Galilei – Ostiglia (MN)

Il controllo degli errori in trasmissione

Durante la trasmissione, il segnale può essere alterato da:

- rumore elettrico;
- interferenze;
- distorsioni.

Per questo motivo i dati ricevuti possono non corrispondere a quelli inviati. Per permettere al destinatario di accorgersi dell'errore, il mittente aggiunge ai dati dei **bit di ridondanza**.Questi bit consentono di costruire **codici di controllo**, che possono essere:

- **rilevatori**: segnalano che c'è un errore;
- **correttori**: permettono anche di individuare il bit errato e correggerlo.

Se i dati sono m bit e i bit ridondanti sono r, la sequenza trasmessa prende il nome di **codeword** e ha lunghezza:

$$n = m + r$$

Distanza di Hamming

Definizione

Date due codeword valide della stessa lunghezza, la **distanza di Hamming** è il numero di bit in cui differiscono nella stessa posizione.

Esempio

$$101101 \qquad 100111$$

Le due codeword differiscono in 2 posizioni:

$$d = 2$$

Distanza di Hamming di un codice

È la **distanza minima** tra tutte le coppie di codeword valide del codice.

Significato della distanza di Hamming

Idea fondamentale

Se due codeword hanno distanza d, servono almeno d errori di bit per trasformarne una nell'altra.

Interpretazione

- distanza bassa $\Rightarrow$ codeword vicine $\Rightarrow$ facile confonderle
- distanza alta $\Rightarrow$ codeword lontane $\Rightarrow$ più sicurezza

A cosa serve

La distanza di Hamming determina la capacità del codice di:

- rilevare errori
- correggere errori

Rilevazione e correzione degli errori

Rilevazione degli errori Per rilevare fino a k errori: $d = k + 1$

- $d = 2 \Rightarrow$ rileva 1 errore
- $d = 3 \Rightarrow$ rileva fino a 2 errori

Correzione degli errori Per correggere fino a k errori: $d = 2k + 1$

- $d = 3 \Rightarrow$ corregge 1 errore
- $d = 5 \Rightarrow$ corregge 2 errori

Sintesi:
Maggiore distanza $\Rightarrow$ maggiore affidabilità

ma anche $\Rightarrow$ più bit ridondanti e codeword più lunghe

Codici di parità

I **codici di parità** hanno distanza di Hamming pari a 2: permettono quindi di **rilevare un errore singolo**, ma non di correggerlo.

Alla sequenza di bit si aggiunge un solo bit di controllo, in modo che il numero totale di 1 sia:

- **pari** (*parità pari*);
- **dispari** (*parità dispari*).

Esempio: se nella sequenza ci sono già un numero pari di bit 1,

- in parità pari si aggiunge 0;
- in parità dispari si aggiunge 1.

Codici di parità

Il ricevente ricalcola la parità e la confronta con quella ricevuta:

- se è diversa, c'è stato un errore;
- se coincide, la sequenza è probabilmente corretta.

Limite: non rileva gli errori doppi e non individua la posizione del bit errato.

Sia data la sequenza di bit: 01100010101111

Il numero di bit "1" è 8, quindi pari, allora il bit di parità sarà 0 per parità pari e 1 per parità dispari:

parità pari: 011000101011110

parità dispari: 011000101011111

Il ricevitore provvederà a ricalcolare il bit di parità sulla sequenza di bit ricevuta, escluso il bit di parità aggiunto, e confronterà il bit di parità ottenuto con quello ricevuto: se sono diversi la trasmissione non è avvenuta correttamente, se invece sono uguali è probabile che la sequenza ricevuta sia quella originale; tuttavia, poiché questa tecnica non rileva gli errori doppi, non se ne ha la garanzia assoluta.

Codici di ridondanza ciclica (CRC)

Il **CRC** (*Cyclic Redundancy Code*) è uno dei metodi più usati nelle reti per la **rilevazione degli errori**.

Il procedimento si basa su un **polinomio generatore** $G(x)$, noto sia al mittente sia al ricevente.

1. I bit del messaggio vengono interpretati come coefficienti di un polinomio $M(x)$.
2. Si aggiungono in coda alcuni 0.
3. Si divide in modulo 2 per $G(x)$, usando l'operazione XOR.
4. Il resto della divisione costituisce il **CRC**.

Il ricevente ripete la divisione:

- se il resto è nullo, la sequenza si considera corretta;
- se il resto non è nullo, si rileva un errore.

Importante: il CRC rileva l'errore, ma non lo corregge.

Esempio CRC - Parte 1

Supponiamo di dover trasmettere la sequenza di bit dati: 101010101101 e di usare come polinomio generatore **G(x)** = $x^3 + x + 1$.

$M(x) = x^{11} + x^9 + x^7 + x^5 + x^3 + x^2 + 1$ è il polinomio corrispondente alla sequenza di bit dati

G(x) = $x^3 + x + 1$ è un polinomio di grado 3 che corrisponde alla stringa di bit: 1011

Dovendo procedere con la divisione, si aggiunge in coda alla sequenza di bit dati un numero di 0 pari al grado del polinomio generatore, quindi il dividendo diventerà la stringa: 101010101101000 corrispondente a **D(x)**, mentre il divisore sarà la stringa

Esempio CRC - Parte 2

1011, corrispondente a **G(x)**. La divisione in modulo 2 serve a calcolare il **resto** della divisione:

```
1 0 1 0 1 0 1 0 1 1 0 1 0 0 0   :   1 0 1 1
1 0 1 1
0 0 0 1 1 0 1
      1 0 1 1
      0 1 1 0 0
        1 0 1 1
        0 1 1 1 1
          1 0 1 1
          0 1 0 0 1
            1 0 1 1
            0 0 1 0 0 1
                1 0 1 1
                0 0 1 0 0 0
                    1 0 1 1
                    0 0 1 1 0        ==>   C R C  =  1 1 0
```

La sequenza di bit che viene trasmessa sul canale diventa: 101010101101110.

Codici di Hamming

Definizione

I codici di Hamming sono codici aventi distanza di Hamming minima uguale a:

$$d = 3$$

Conseguenza

Poiché per correggere k errori vale:

$$d = 2k + 1$$

con $d = 3$ si ottiene:

$$3 = 2k + 1 \Rightarrow k = 1$$

Nota Bene:

I codici di Hamming permettono quindi di **correggere automaticamente un errore singolo**.

Bit di dati e bit di controllo

Una codeword lunga n bit è formata da:

- m bit di dati;
- r bit ridondanti di controllo, chiamati anche **bit di parità** o **bit di check**.

$$n = m + r$$

Regola di Hamming

La tecnica di Hamming impone:

$$2^r - 1 \geq m + r$$

Esempi limite

$$r = 3 \Rightarrow m = 4 \Rightarrow n = 7$$

$$r = 4 \Rightarrow m = 11 \Rightarrow n = 15$$

$$r = 5 \Rightarrow m = 26 \Rightarrow n = 31$$

Posizione dei bit di check

Regola generale

I bit di check vengono inseriti nelle posizioni corrispondenti alle potenze del 2:

$$b_1, \quad b_2, \quad b_4, \quad b_8, \quad b_{16}, \quad \ldots$$

Significato

- con b_1 e b_2 si controllano posizioni fino a b_3;
- aggiungendo b_4 si controllano posizioni fino a b_7;
- aggiungendo b_8 si controllano posizioni fino a b_{15}.

Posizione dei bit di check

Codeword			
Posizione	b_1	b_2	b_3

Codeword							
Posizione	b_1	b_2	b_3	b_4	b_5	b_6	b_7

Codeword															
Posizione	b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}	b_{11}	b_{12}	b_{13}	b_{14}	b_{15}

Inserimento dei bit di dati

Vogliamo trasmettere 11 bit di dati:

$$11100111010$$

Codeword da 15 bit

Servono 4 bit di controllo:

$$b_1, \quad b_2, \quad b_4, \quad b_8$$

per ottenere una codeword complessiva lunga 15 bit.

Procedura

- si lasciano libere le posizioni $1, 2, 4, 8$;
- si inseriscono i dati nelle altre posizioni:

$$b_3, b_5, b_6, b_7, b_9, b_{10}, b_{11}, b_{12}, b_{13}, b_{14}, b_{15}$$

Inserimento dei bit di dati

1 1 1 0 0 1 1 1 0 1 0

Codeword			1		1	1	0		0	1	1	1	0	1	0
Posizione	b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}	b_{11}	b_{12}	b_{13}	b_{14}	b_{15}

Servono dunque 4 bit di controllo ($\mathbf{b_1}$ $\mathbf{b_2}$ $\mathbf{b_4}$ $\mathbf{b_8}$) per un totale di codeword di 15 bit.

Quali posizioni controllano i bit di check

Tabella dei controlli

Per calcolare i bit di check si usa la seguente logica:

Bit di check	Posizioni controllate
b_1	$3, 5, 7, 9, 11, 13, 15$
b_2	$3, 6, 7, 10, 11, 14, 15$
b_4	$5, 6, 7, 12, 13, 14, 15$
b_8	$9, 10, 11, 12, 13, 14, 15$

Esempio

La posizione 6 viene controllata da:

$$b_2 \quad \text{e} \quad b_4$$

perché 6 compare nelle righe di b_2 e b_4.

Spiegazione della tabella di controllo

Viene usata l'dea dei pesi

Ogni posizione può essere letta come somma di potenze di 2.

Esempio: posizione 13

La posizione 13 in binario vale:

$$13 = 1101_2$$

e può essere scritta come:

$$13 = 1 + 4 + 8$$

Conseguenza

La posizione 13 viene quindi controllata dai bit:

$$b_1, \quad b_4, \quad b_8$$

e non da b_2.

Nota Bene: poiché non esistono due numeri diversi con gli stessi pesi, la combinazione dei bit di check individua **univocamente** la posizione errata.

Calcolo dei bit di check

Supponiamo di usare la **parità pari**.

Otteniamo quindi

:

$$b_1 : 1, 1, 0, 0, 1, 0, 0 \Rightarrow b_1 = 1$$

$$b_2 : 1, 1, 0, 1, 1, 1, 0 \Rightarrow b_2 = 1$$

$$b_4 : 1, 1, 0, 1, 0, 1, 0 \Rightarrow b_4 = 0$$

$$b_8 : 0, 1, 1, 1, 0, 1, 0 \Rightarrow b_8 = 0$$

Codeword finale

Inserendo i bit di check calcolati:

$$\boxed{1\ 1\ 1\ 0\ 1\ 1\ 0\ 0\ 0\ 1\ 1\ 1\ 0\ 1\ 0}$$

Codeword	**1**	**1**	**1**	**0**	**1**	**1**	**0**	**0**	**0**	**1**	**1**	**1**	**0**	**1**	**0**
Posizione	b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}	b_{11}	b_{12}	b_{13}	b_{14}	b_{15}

Controllo in ricezione

Il destinatario quindi:

- riceve la codeword;
- ricalcola i bit di check con la stessa tabella;
- confronta i bit ricalcolati con quelli ricevuti tramite XOR.

Interpretazione dello XOR sul controllo:

- se il risultato è formato solo da zeri, non ci sono errori;
- se il risultato è diverso da zero, il numero binario ottenuto indica la posizione dell'errore.

Ipotizziamo un errore in posizione 13.

Codeword	1	1	1	0	1	1	0	0	0	1	1	1	1	1	0
Posizione	b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9	b_{10}	b_{11}	b_{12}	b_{13}	b_{14}	b_{15}

Individuazione della posizione errata

Con l'errore in posizione 13, il ricalcolo produce:

$$b_1 = 0$$

$$b_2 = 1$$

$$b_4 = 1$$

$$b_8 = 1$$

Confronto XOR (utilizzando l'ordine di peso):

b_8	b_4	b_2	b_1
1	1	1	0
0	0	1	1
1	1	0	1

Risultato:

$$1101_2 = 13$$

Quindi l'errore si trova nella posizione: b_{13}

Correzione e limiti

Correzione

Una volta individuata la posizione errata, il destinatario corregge il bit invertendolo:

$$0 \leftrightarrow 1$$

senza chiedere la ritrasmissione della codeword.

Limite principale:

La tecnica illustrata corregge solo **errori singoli**.

Errori doppi

Gli errori doppi non vengono corretti correttamente: possono essere interpretati come un errore singolo in un'altra posizione.

Più bit di controllo permettono maggiore affidabilità, ma:

- aumentano la lunghezza della codeword;
- riducono il throughput;
- occupano più canale trasmissivo.